

Document 09 — LangGraph

This is for self-study and live mentoring. Full plan: [CURRICULUM.md](#)

Prerequisites

[08_rag](#) + [Async Python gate](#)

How to Read & Practice This Document

- **What:** building an agent as a clear, visible graph instead of a hidden loop.
- **Why:** real systems need state you can look at, test, and pause — not a black-box `while` loop you can't inspect or replay.
- **When:** any agent complex enough to need branching, saved state, or a human checking in — Project 2's simple loop genuinely didn't need this; Project 3 will.
- **How to practice:** 1. Read the LangGraph quickstart, then go straight to the `examples/` folder in the repo — this tool is best learned by reading real graphs, not just prose. 2. Do the **Basic/Intermediate** graph exercises closed-book where you can. 3. Rebuild Project 2 as a graph (**Real-world**) — same behavior, new build. Don't add new features until the rebuild is proven correct. 4. Do the **Failure/Production** exercises on purpose — an intentionally-broken loop and a real pause/resume test teach more than reading about them. 5. Use **Hint 1** → **Hint 2** only after really trying. Ask for the full solution only if you say "**Show me the solution.**" 6. Before moving on, draw the graph on paper from memory and check it against your code. If they don't match, you're not done.

Jump to: [Core Concepts](#) · [Practice Exercises](#) · [Build Task](#)

The Story — what this document is actually building

Picture this: Project 2's agent (Doc07) works, but it's a black box. It's a `while` loop, hidden inside library code, deciding what to do next — you can't easily look at one single step, test it by itself, or pause it partway through and pick up again later. That's fine for a demo. It's not fine for anything you'd actually want to run, debug, and trust in production.

LangGraph's whole idea is to make that hidden loop *visible* as a graph you build yourself, out of code you own. **State** is one typed shape holding everything the graph is tracking — the task, the conversation, whatever's been found so far. **Nodes** are just functions: state goes in, a small update

comes out. **Edges** say what runs next — a plain edge always goes to the same place, a **conditional edge** picks based on something in state, which is how "should I search or not" branching gets written as real, visible structure instead of a buried `if`. Graphs are even allowed to **loop** back on themselves on purpose — that's how "keep improving until it's good enough" gets built, as long as there's a real stopping rule.

Then two more pieces make this genuinely production-ready instead of just a nicer diagram. A **checkpointer** saves the graph's state at every step, so it survives outside any one running program — Doc04's chatbot memory disappeared the moment you closed the terminal; a checkpointed graph's state doesn't. And `interrupt()` uses that saved state to pause the graph mid-run and wait for a human — approve this refund, confirm this email gets sent — before continuing exactly where it left off.

The **Build Task** ties all of this together: you rebuild Project 2's exact tool-using behavior as a graph (proving you can translate a loop into nodes and edges), add a conditional edge, wire up a checkpointer, and add one real `interrupt()` point. This isn't a throwaway exercise — it's the literal skeleton that Document 10 builds into the full Project 3 app.

Core Concepts (read this first — everything you need is here)

Topics on this page: [LangChain vs. LangGraph vs. RAG](#) · [From a hidden loop to a clear graph](#) · [State](#) · [Nodes and edges](#) · [Loops on purpose](#) · [Checkpointers](#) · `interrupt()` · [Parallel branches](#) · [Subgraphs](#) · [Streaming a graph's execution](#) · [Debugging a graph visually](#) · [Long-term memory](#)

LangChain vs. LangGraph vs. RAG — how these three actually relate

What each one actually is, in one line: LangChain (Doc05) is a library of building blocks for calling a model — prompt templates, `|`-chained pipelines (LCEL), output parsers. LangGraph (this document) is a different kind of tool from the same team — a graph-based way to structure *multi-step, possibly-looping* logic, with real state, branching, pausing, and memory. RAG (Doc08) isn't a library at all — it's a *technique*: retrieve relevant text, then generate an answer grounded in it.

Why people mix these up: all three showed up in this curriculum around the same time, and all three involve "calling a model," so it's easy to think of them as competing options you pick one of. They're not on the same axis. LangChain and LangGraph both answer "how do I structure the code that calls the model" — LangChain for a straight-line pipeline (prompt → model → parser, or a single non-looping agent), LangGraph for something that needs to branch, loop on purpose, pause for a human, or remember across separate runs. RAG answers a completely different question — "how do I ground the model's answer in my own real data" — and it doesn't care which of the other two you use to build it: Doc08's Build Task builds RAG with plain Python, and Project 3 wires that exact same `retrieve()` function in as one node of a LangGraph graph.

When to reach for which: | Situation | What fits | |---|---| | One prompt, one model call, maybe a parser — no branching, no loop | LangChain (or just the raw SDK, per Doc02/04's own comparison) |

| A single agent that calls tools in a loop, but never needs to pause or persist state across restarts | LangChain's `create_agent` (Doc07) is enough | | Multiple steps that branch, loop on purpose, need to pause for human approval, or must survive a process restart | LangGraph — this is exactly why Doc10's search agent and every multi-agent project (4, 9, 10) are built on it, not on LangChain alone | | The model needs facts from your own documents, regardless of how the rest of the app is structured | RAG (Doc08) — build it as a plain function, then call that function from wherever you need it, LangChain chain or LangGraph node |

How they actually combine in this curriculum's own projects: Project 3 is the clearest example — a LangGraph graph (the orchestration) with one node that runs Doc08's RAG retriever (the grounding technique) and other nodes built using LangChain pieces (Doc05's prompt templates) for the actual model calls. None of the three replaced either of the others — each one is solving a different part of the same problem.

Could you just do all of this in LangChain instead? Capability by capability:

- **Branching on the model's decision.** LangChain's LCEL has `RunnableBranch` for picking between a few fixed paths based on a condition. *Why it struggles beyond that:* it's built for a one-shot decision, not a decision that itself needs its own state or might route back to an earlier step — chaining several `RunnableBranch` es together to simulate a real graph gets unreadable fast, because LCEL's `|` syntax assumes data flows one direction, forward. *How you'd hack it anyway:* wrap each branch's logic in its own chain, and use a plain Python `if/elif` around calling `chain_a.invoke(...)` vs `chain_b.invoke(...)` — at which point you've essentially reinvented a graph's conditional edge by hand, in regular code, which is exactly the "hidden inside library internals vs. visible as code you own" trade-off from the topic above.
- **Loops on purpose (try again, ask again, refine again).** LCEL has no native "run this chain again based on its own last output" primitive — chains are built to run once, start to finish. *Why it struggles:* a loop needs to check a condition after each run and decide whether to stop or go again, which isn't something you express by connecting `|`-chained pieces; you'd need to step outside LCEL. *How you'd hack it anyway:* a plain Python `while` loop calling `chain.invoke(...)` repeatedly, checking the result yourself each time — again, correct, but now the "loop logic" lives in bespoke Python, not in anything LangChain understands or can help you inspect, checkpoint, or visualize.
- **Pausing for a human mid-task, then resuming exactly where it left off (`interrupt()`).** LangChain has no equivalent — a chain that's `.invoke()` d runs to completion or raises an error; there's no built-in way to freeze execution *between* steps and hand control back to your code, then continue from that exact point later. *How you'd hack it anyway:* stop the chain after the step before the human decision, save whatever state you'd need to resume (by hand, to your own database or file), and write a second script that reloads that state and continues — you're manually rebuilding what LangGraph's checkpointing does for you, and it's easy to get subtly wrong (forgetting one piece of state that turns out to matter later).

- **Surviving a process restart mid-task (checkpointing).** Same root problem as pausing — LCEL chains keep their state in memory, in Python variables, for the life of one `.invoke()` call. *How you'd hack it anyway:* serialize your own state to a file or database after every step, and write your own resume logic that reconstructs where you were — this is a real, working pattern (plenty of production code did exactly this before LangGraph existed), it's just code you now own and have to maintain and debug yourself, instead of `SqliteSaver` handling it.
- **Running two things at once and merging results (parallel branches).** LCEL actually has a real answer here — `RunnableParallel` runs multiple chains concurrently and merges their outputs into one dict. *Where it stops being enough:* it merges results once, at one fixed point — it doesn't handle the more general graph case of several *different* nodes conditionally running in parallel some of the time, sequentially other times, based on runtime decisions, which is what LangGraph's fan-out/fan-in supports.
- **Remembering a fact across separate, later conversations.** LangChain has no built-in long-term store. Its older memory classes (like the now-removed `ConversationBufferMemory`) were scoped to one conversation, similar to a LangGraph checkpointer's per-thread state, not across threads — and current LangChain code manages a conversation's messages directly as a plain list instead, which is still just per-conversation. *How you'd hack long-term memory anyway, without LangGraph:* build your own tiny key-value store (a JSON file, a SQLite table, a real database) keyed by user ID, write to it yourself when something worth remembering happens, and read from it yourself at the start of each new conversation — functionally similar to `langgraph.store`, just without the namespacing/search conveniences built in.

The same comparison, side by side: | Capability | LangChain's answer | LangGraph's answer | |---|---| | Branch on a decision | `RunnableBranch` — one fixed decision point, no easy way back | A conditional edge — any node can route to any other, including back to an earlier one | | Loop until done | No native primitive — a manual `while` loop around `.invoke()` | A real edge back to an earlier node — the loop is part of the graph itself, visible and inspectable | | Pause for a human, resume later | No native primitive — you save/reload state by hand | `interrupt()` + a checkpointer — the framework saves and restores state for you | | Survive a process restart | No native primitive — you serialize state yourself | A checkpointer (`SqliteSaver`, etc.) — persistence built in | | Run steps in parallel | `RunnableParallel` — merges a fixed set of chains once | Fan-out/fan-in — any number of nodes, conditionally, merged via a reducer | | Remember across separate conversations | No native primitive — build your own key-value store | `langgraph.store` — a namespaced, built-in long-term store |

The honest summary: none of this is *impossible* in LangChain — every one of these patterns existed in real production code before LangGraph did, built the hard way, by hand, in plain Python around LangChain's chains. What LangGraph actually adds is turning each of those hand-built workarounds into a first-class, visible, reusable piece of the framework — which is why this document exists as its own thing, not as one more LangChain feature.

Tool-calling inside a graph, and the path from one agent to many

The story — what we're actually doing here: Project 2 built one agent that calls tools in a loop, hand-built, and compared it to LangChain's `create_agent`. Now imagine the task has grown — you don't need one generalist anymore, you need a researcher and a separate fact-checker, each with their own tools and their own job. This topic is the bridge between those two moments: what actually changes when you go from "one tool-calling agent" to "several," and — just as importantly — what *doesn't* change.

What tool-calling inside a graph node actually looks like: exactly the same mechanism Doc06 and Doc07 already taught — a model call with a list of registered tools, a check for `tool_calls` in the reply, running the tool, feeding the result back. The only difference is *where* that logic lives: instead of being the whole program (Doc07's `create_agent` loop), it's the body of one node in a graph, using LangGraph's `ToolNode` helper or a small hand-written function that does the same job.

Why this matters — the one thing to really understand: the tool-calling code itself does not get more complicated as you add more agents. `researcher_node` and `fact_checker_node` in a multi-agent graph (Doc11's territory) each look almost exactly like Doc07's single-agent loop, just shorter — one call, one tool-check, return an update. What grows is the graph *around* them: more nodes, more edges, a router deciding who goes next. Multi-agent isn't "harder tool-calling" — it's the same tool-calling, repeated, with a routing layer added on top.

When you actually need more than one tool-calling node: exactly when Doc11's "The real question: one agent, or many?" topic says to — genuinely different specialties, or a task large enough that one system prompt covering everything gets confused. Not before. A single `ToolNode`-equipped agent, wrapped in the smallest possible graph (one node, no branching), is still the right answer for most tasks — this document's own Build Task is exactly that: one graph, one tool-calling step, no multi-agent complexity yet.

How this connects back to LangChain, concretely: the tool list and the tool-calling logic you'd write are *identical* whether they end up inside a plain LangChain `create_agent` (Doc07, one agent, no graph) or inside a LangGraph node (this document, one agent as a graph, or many agents as a bigger graph). Nothing about the tools changes — only the orchestration wrapped around them does, which is the exact same distinction the comparison above draws for every other capability.

From a hidden loop to a clear graph

Doc07's `create_agent` loop hides how it decides what to do next, inside library code — you can't easily see, test, or change one single step without reading the library's internals. A `StateGraph` makes every step visible: you define nodes (units of work) and edges (what runs next) yourself, as code you own, can read, and can test one piece at a time. **Why this is the whole point of LangGraph, not just a style choice:** real systems need to be checkable (what actually happened during this run?), testable (does this one piece work correctly by itself?), and resumable (can we pause and pick up later?) — a hidden loop can't give you any of that. A clear graph can, because every step is a real, findable piece of code.

State: one typed object flowing through the graph

The graph's **state** is one shape (a `TypedDict` or Pydantic model) that lists every piece of data that can move between steps — the current task, the conversation so far, anything found by searching, whatever the graph needs to track. Each step (node) gets the current state, and returns a small update to it, which LangGraph merges in before passing the updated state to the next step. **Why one typed shape matters:** it makes the whole graph's data flow visible in one place — you can read the state shape and know exactly what any step could possibly see or change, instead of tracing scattered variables through hidden loop code.

Nodes and edges, including conditional edges

A **node** is just a function: state goes in, a small update comes out — it does one piece of work (call a tool, call the model, check something) and nothing more. An **edge** connects nodes, saying what runs next. A plain edge always goes to the same next node. A **conditional edge** instead runs a small function against the current state and picks the next node based on the answer — this is how branching logic (search or don't, approve or reject) gets written as real graph structure, instead of buried inside a big if/else deep in one function.

Loops: on purpose, not by accident

Unlike a typical one-way flowchart, LangGraph graphs are allowed to loop back on themselves — a node's conditional edge can send things back to an earlier node. This is exactly how you build "keep trying/improving until something's good enough" behavior (this becomes central in Doc11's writer-and-critic pattern). **Why this needs a real stopping rule, just like Doc07's agent loop did:** a loop with no real exit condition is exactly as dangerous as a missing step limit — the graph structure doesn't protect you from an endless loop by itself, it just makes the loop *visible* in the diagram instead of hidden inside library code.

Checkpointers: saved state that survives a pause

A **checkpointer** saves the graph's state at each step (kept in memory while testing, in a real database for production) — this is what makes pausing and resuming possible. Without one, a graph that stops has simply lost all its state. **Why this is something Doc04's chatbot never had:** that chatbot's "memory" only ever existed as a Python list inside a running program — kill the program, and it's gone. A saved graph's state exists on its own, separate from any one running program — which is the base for both human-checkpoint pauses (below) and a chatbot that remembers you across separate sessions *of the same conversation thread*. A genuinely new conversation is a different story — see [Long-term memory: remembering across separate conversations](#) below.

`interrupt()` : pausing for a human

Calling `interrupt()` inside a node pauses the graph right there, saves the state, and hands control back to whatever's running the graph — later, a separate call resumes it from exactly where it

paused, using the saved state. **Why this matters for building real agents:** some actions (sending an email, spending money, deleting data) are important enough that you want a human to approve them before the graph continues. `interrupt()` makes that a real, built-in part of the graph, instead of something you'd have to hack together with flags and polling.

Parallel branches (fan-out / fan-in)

A graph doesn't have to run one node after another in a single line — you can send one edge out to two or more nodes at once (a "fan-out"), let each one do its own work, and then point all of them forward to the same next node so their results come back together (a "fan-in"). **Why this matters:** if a task needs two independent lookups — say, checking a price API and checking a weather API — running them one after another wastes time waiting for the first to finish before the second even starts; running them in parallel means both are in flight at the same time, and the graph only waits as long as the slower one takes. **How it works:** you connect one node to two (or more) other nodes with plain edges, and connect all of those forward to one shared node — LangGraph runs the parallel nodes concurrently and merges their state updates before the shared node runs. **The state-merging question this raises:** if two parallel branches both try to update the *same* field in state, which update wins? By default a plain field just gets overwritten, so the last update to arrive silently wins and you lose the other one — this is exactly why the reducer functions from the **State** topic above matter here. Give a field a reducer (like `operator.add`, or a custom merge function) and both branches' updates get combined instead of one clobbering the other.

```
graph.add_edge("start", "lookup_price")
graph.add_edge("start", "lookup_weather")
graph.add_edge("lookup_price", "combine_results")
graph.add_edge("lookup_weather", "combine_results")
```

Subgraphs: a graph as one node in a bigger graph

A **subgraph** is a whole compiled `StateGraph` used as a single node inside a bigger graph — from the outside, the bigger graph just sees one node that takes state in and returns an update, even though internally that "node" is quietly running its own multi-step graph. **Why you'd do this:** the same reason Doc01 pulled config and logging out into their own files instead of leaving everything in one script — a smaller, self-contained piece is easier to test on its own, easier to reuse in more than one place, and easier to reason about without holding the whole system in your head at once. This document's own Build Task skeleton is a good candidate: instead of copy-pasting its nodes into a bigger graph later, you can wrap the whole skeleton as one subgraph node and plug it straight in, already tested and correct. **How it works:** you compile the smaller graph exactly as normal — `smaller_graph = builder.compile()` — then add it to the bigger graph the same way you'd add any function node: `bigger_builder.add_node("build_task", smaller_graph)`. This works as long as the smaller graph's state shape and the bigger graph's state shape share the fields that need to pass between them.

Streaming a graph's execution

`.invoke()` runs the whole graph and only hands you a result once everything is finished — fine for a script, but a poor fit for anything a person is watching happen live. `.stream()` instead gives you each node's output as it happens, one step at a time, while the graph is still running. **Why this matters:** a chat UI that shows "searching your documents... thinking... writing an answer..." step by step needs to know what's happening *during* the run, not just the final answer at the end — without streaming, the user just stares at a blank screen for however long the whole graph takes. **How it works:** instead of `result = graph.invoke(state)`, you loop over the stream: each item is the update produced by whichever node just finished running, so you can show progress, log intermediate state, or update a UI element the moment each node completes, instead of waiting for the entire graph.

```
for step in graph.stream({"task": "find the invoice total"}):
    print(step)
```

Debugging a graph visually

LangGraph can draw an actual picture of the graph you built — its nodes, its edges, and which edges are conditional — instead of you having to read through a pile of `add_node / add_edge` calls and mentally reconstruct the shape in your head. **Why this matters:** a graph with a wrong or missing edge is usually much faster to spot by looking at the diagram than by reading code line by line — a node with no arrow leaving it, or an edge pointing to the wrong place, jumps out visually in a way it doesn't in text. **How it works:** `graph.get_graph().draw_mermaid()` returns a Mermaid diagram (text you can paste into a Mermaid renderer, or into most Markdown viewers that support it) showing every node and edge in your compiled graph — there's also a `draw_mermaid_png()` option if you want an image file directly. Reach for it any time a graph isn't behaving the way you expect; it's often the fastest way to answer "is this graph actually shaped the way I think it is?"

```
print(graph.get_graph().draw_mermaid())
```

Long-term memory: remembering across separate conversations

A **checkpoint** (above) persists *one conversation's* state so that same conversation can pause and resume — but a brand new conversation (a new thread ID) still starts with a brand new, empty state, checkpoint or not. **What long-term memory actually is:** a separate store that survives *across* different conversations entirely — the same user, talking to the agent again next week in a totally new thread, and the agent still remembers their name or a preference they stated last time. **Why this needs its own mechanism:** a checkpoint is keyed by thread ID on purpose, so one user's in-progress conversation doesn't leak into another's — long-term memory has to live outside that, keyed by something that outlives any one thread, like a user ID. **How it works:** LangGraph's `Store` interface (`langgraph.store.memory.InMemoryStore` for local dev,

`langgraph.store.postgres.PostgresStore` for production) holds items under a **namespace** — usually something like `(user_id, "memories")` — and a node can call `store.put(namespace, key, value)` to save a fact, `store.get(namespace, key)` to fetch one, or `store.search(namespace, query=...)` to find relevant ones, completely independent of whatever thread the current run belongs to. A node typically reads from the store at the start of a run (to load what's already known about this user) and writes to it during or after the run (to save anything new worth keeping).

```
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()
namespace = (user_id, "memories")
store.put(namespace, "name_preference", {"text": "Prefers to be called Sam"})

remembered = store.search(namespace, query="how should I address this user?")
```

The real design question: what's worth saving forever, versus what should just stay in the current conversation? You don't want to save every message ever said — a full transcript of every past conversation dumped back into every future run is neither useful nor readable — you want only genuinely durable facts: a name, a stated preference, a past decision. **Deciding what to save is itself often a small job for the model:** after a message, a small LLM call can ask something like "does this message contain a fact worth remembering long-term?" and only write to the store when the answer is yes — the same judgment call a person would make, just automated as one more (cheap) model call in the flow.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- [LangGraph documentation \(home\)](#) — start with the Quickstart/tutorials.
- [LangGraph GitHub repo](#) — the `examples/` folder has real graphs to read.
- [LangChain Academy](#) — has a free LangGraph course; do it alongside this document.

Practice Exercises

Setup for this document's practice code: work inside `09_langgraph/` (same venv as before — if it's not active, `source .venv/bin/activate`). New package for this document: `pip install langgraph langchain-openai`.

How to run each exercise: group your practice code by topic, not by difficulty level. If two exercises below are really about the same thing, save them together in ONE script named after that topic —

for example, if two exercises are both about `.env` config, save both in one file like `env_config_practice.py`, with each level's version as its own clearly labeled section inside it. Run each topic's file directly, for example: `python env_config_practice.py`.

For this document: - Basic (`first_graph`) is its own topic — save it as `first_graph_practice.py`. - Intermediate (`conditional_routing`) and Edge cases (`unhandled_routing_value`) are both about how conditional edges route between paths — save them together as `conditional_routing_practice.py`, with each level as its own section. - Real-world (`agent_loop_to_graph`) is its own topic — save it as `agent_loop_to_graph_practice.py`. - Failure (`loop_limit_and_interrupt_resume`) is its own topic — save it as `loop_limit_interrupt_practice.py`.

Why group by topic instead of by level: if you save each exercise by difficulty level instead, the different versions of the same idea end up scattered across separate files, and you can never see how one topic grows from simple to harder in one place. Grouping by topic keeps that growth visible — open one file, and you see the whole journey for that one thing, from basic to advanced, side by side.

Jump to an exercise: [Basic](#) · [Intermediate](#) · [Real-world](#) · [Edge cases](#) · [Failure](#) · [Build Task](#)

Basic — your first graph

- **What:** a 2-node `StateGraph` with no branching, tracing by hand what the state looks like after each node.
- **Why:** you need the simplest possible graph working, with a mental model you've checked by hand, before adding anything that could hide a misunderstanding.
- **When you'll hit this for real:** the very first LangGraph code you'll ever write, right here.
- **How to code it:** define a `TypedDict` state with one field, two node functions that each update it, `graph.add_node(...)` for both, `graph.add_edge(...)` connecting them, then `graph.compile().invoke({...})` and print the result.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Intermediate — real branching

- **What:** a conditional edge that picks a path based on a field in state, tested on both paths.
- **Why:** this is the mechanism every "should I search / should I call this agent" decision in the rest of the curriculum is built from.
- **When you'll hit this for real:** Project 3's "search or don't" routing, and Project 4's supervisor routing.
- **How to code it:** write a function `def route(state): return "path_a" if state["flag"] else "path_b"`, wire it with `graph.add_conditional_edges("node", route, {"path_a": "...", "path_b": "..."})`, and invoke with both flag values.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Real-world — rebuild Project 2 as a graph

- **What:** rebuild Project 2's agent (Doc07) as a graph with the same tools — same behavior, a new, clearer build.
- **Why:** this is the exact translation exercise (loop → graph) this document's Build Task needs — do a first pass here, on a system you already understand completely.
- **When you'll hit this for real:** this document's own Build Task, feeding directly into Project 3.
- **How to code it:** turn your Doc07 loop's think/act/observe steps into 2-3 nodes, with a conditional edge deciding "call a tool again" vs. "finish," and confirm it gives the same answers as your original loop on the same test prompts.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Edge cases — an unhandled routing value

- **What:** a conditional edge whose check function returns a value that doesn't match any known path — see what happens.
- **Why:** you want to know, before it happens in a real run, whether this fails loudly (good) or silently does something unexpected (bad) — and if it's the latter, you need to guard against it yourself.
- **When you'll hit this for real:** a routing function with a bug, or a state value you didn't account for — this will happen to you eventually.
- **How to code it:** deliberately have your routing function sometimes return a string not in your `add_conditional_edges` mapping, run it, and read exactly what LangGraph does.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Failure — an endless loop, and a real pause/resume

- **What:** a graph with a loop that has no exit rule, run in a controlled, limited way. Then a checkpointer connected, proving state survives an `interrupt()` → resume cycle.
- **Why:** both of these are things you need to have actually watched happen once — an endless graph loop, and a real, working pause that resumes correctly — before you trust yourself to build them safely in a real project.
- **When you'll hit this for real:** Project 3's own `interrupt()` gate, and any generator ↔ critic loop in Project 4.
- **How to code it:** build a 2-node cycle with no conditional exit, run it with a hard recursion limit set low so it fails fast instead of hanging. Then, separately, add `checkpointer=MemorySaver()`, call `interrupt()` in one node, invoke once (it pauses), then resume with the same thread ID and confirm it continues correctly.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Build Task — Graph Skeleton (feeds into Project 3)

Stuck on the Build Task? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Goal: the graph foundation that Document 10 will build into the full Project 3 app.

Requirements:

- A `StateGraph` with a typed state shape.
- At least one conditional edge (not just a straight line of nodes).
- A checkpointer connected — state has to survive a pause.
- At least one `interrupt()` point, standing in for human checking before a "risky" action (the action itself can just be a placeholder for now).
- Rebuilds Project 2's tool-calling behavior as graph nodes (proves you understand turning a loop into a graph).

Inputs: the same free-text task input as Project 2.

Outputs: the same final-answer behavior as Project 2, plus a visible log of the state, and a working pause/resume cycle.

Constraints: no node may quietly swallow an error — failures must show up in state or logs.

Suggested files:

```
09_langgraph/
├─ graph.py
├─ state.py
├─ nodes.py
├─ test_graph.py
```

Functions/Components to build:

- `state.py` → the `TypedDict` /Pydantic state shape
- `nodes.py` → node functions (each takes state, returns a small update)
- `graph.py` → `build_graph()` -> `CompiledGraph` , connecting nodes, edges, checkpointer, and the interrupt

Expected Behavior

- You can draw the node/edge diagram from the code (or the other way around), without running it.
- The graph reaches the same final answers as Project 2, for the same test tasks.
- Pausing at the `interrupt()` point and resuming later (you can fake this with a second script run) gives the correct, continued behavior, using the saved state.

Test Cases

Scenario	Expected
Task with no risky action	The graph finishes without ever hitting the interrupt
Task that reaches the interrupt point	Runs pauses, state is saved
Resume after interrupt (approved)	Graph continues from the saved state, correct final answer
Conditional edge with an unexpected state value	Fails clearly and loudly, not a silent wrong path

Break-It / Debug Preview

- An endless loop from a missing or wrong conditional edge.
- State that doesn't actually survive an interrupt (a badly set up checkpointer).
- A node that changes state in a way that breaks a later node's assumptions.
- Full debugging drill in [14_debugging_lab](#).

Interview Topics Preview

- Why clear state beats hidden agent memory · checkpointer trade-offs (in-memory vs. saved) · when a graph is overkill vs. actually needed vs. a plain `create_agent` loop is enough.

Move On When

You can draw a graph's diagram from its code (or the other way), without running it, and the pause/resume cycle works. Full details: [CURRICULUM.md §4](#).

Stuck? Ask for **Hint 1** through **Hint 2**. Ask for the full solution only if you say "**Show me the solution.**"